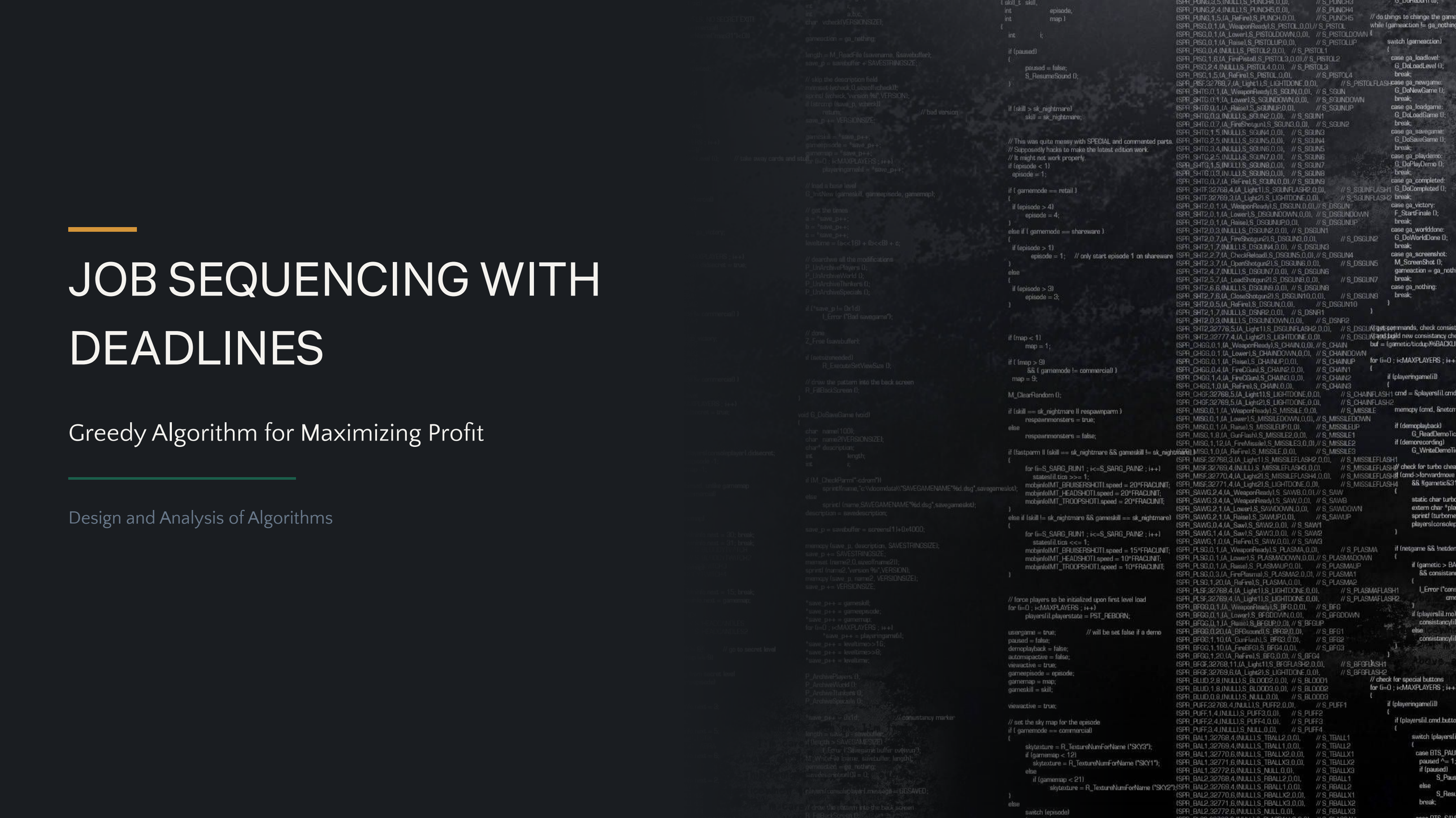
JOB SEQUENCING WITH DEADLINES

Greedy Algorithm for Maximizing Profit

Design and Analysis of Algorithms



Learning Outcomes

By the end of this lecture, students will be able to:

- **Understand** the problem idea and real-life motivation
- **Apply** the greedy strategy and latest slot rule
- **Walk through** a complete worked example step by step
- **Explain** the correctness intuition behind the greedy approach
- **Analyze** time and space complexity of the algorithm

Lecture Roadmap

Real-Life Motivation

Imagine you are a **freelancer** who receives multiple project offers.

Each project has:

A profit amount you will earn

A deadline by which it must be completed

Each project takes **exactly one day** to finish. You can only work on **one project per day**.

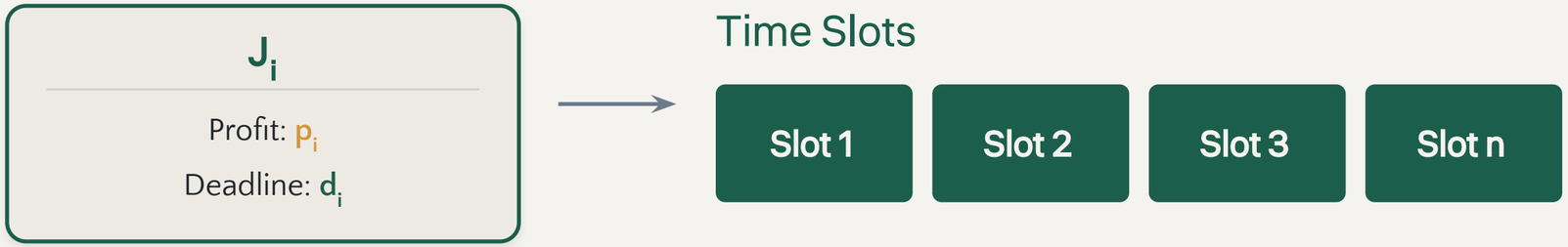
Goal: Choose the optimal set of projects to maximize your total income.

How do you decide which projects to accept and which to decline?

Problem Definition

Given a set of **n jobs**, where each job J_i has a **profit p_i** and a **deadline d_i**, schedule the jobs to **maximize total profit**. Each job takes **1 unit of time**, and profit is earned only if the job is completed **on or before** its deadline.

Job Representation



FORMAL DEFINITION

Input: A set of n jobs, each with profit p_i and deadline d_i ($1 <= d_i <= n$)

Constraint: Each job takes 1 time unit; only one job per time slot

Objective: Find a feasible schedule S that maximizes total profit

Feasible: Job J_i scheduled in slot s where $1 <= s <= d_i$

Key Assumptions

One job = one time unit

Every job takes exactly the same amount of time

One slot = one job

Each time slot can hold at most one job

Deadline is an integer

Deadlines are whole numbers (1, 2, 3, ...)

Late job gives zero profit

No partial credit for missing a deadline

Goal is maximum profit, NOT maximum number of jobs

We prioritize high-value jobs even if it means scheduling fewer total jobs

Why These Assumptions Matter

These assumptions simplify the problem while keeping it practically relevant:

Unit time allows us to treat scheduling as a slot-filling problem

Integer deadlines ensure we have discrete, countable time slots

Profit maximization distinguishes this from simpler scheduling problems

Difference from Activity Selection

Activity Selection

GOAL

Maximize NUMBER of activities

GREEDY RULE

Sort by **earliest finish time**

Pick the activity that ends first

VS

Job Sequencing

GOAL

Maximize TOTAL PROFIT

GREEDY RULE

Sort by **highest profit** first

Place in latest available slot

Key Insight: Different objectives require different greedy strategies. The sorting criterion directly follows from what we are trying to optimize.

Wrong Strategy: Sort by Deadline

INCORRECT

Sorting by **earliest deadline first** may select low-profit jobs and block higher-profit jobs from being scheduled. Here is a counterexample:

Counterexample

Job A	Job B	Job C
Profit: 10	Profit: 50	Profit: 20
Deadline: 1	Deadline: 2	Deadline: 2

WRONG: Deadline Order (A, B, C)

A (10) | B (50) | C skipped

Total Profit = **10 + 50 = 60**

C is blocked because slot 2 is already taken by B

BETTER: Profit Order (B, C, A)

C (20) | B (50) | A skipped

Total Profit = **20 + 50 = 70**

B and C both fit — B in slot 2, C in slot 1. A is low-profit, so skipped.

Correct Greedy Strategy

CORRECT

1

Sort by Profit Descending

Arrange all jobs from **highest profit** to **lowest profit**.

This ensures high-value jobs get **first priority** for scheduling.

2

Place in Latest Available Slot

For each job, find the **latest free slot** at or before its deadline.

If found, assign the job. Otherwise, **skip** it.

This preserves earlier slots for jobs with tighter deadlines.

Jobs Ranked by Profit

#1 Highest Profit Job (process first)

#2 Second Highest Profit (process second)

#3 Third Highest Profit (process third)

#4 Fourth Highest Profit (process fourth)

#n Lowest Profit Job (process last)

High-profit jobs get the first chance at every available slot

Why Profit First?

Because the **OBJECTIVE IS PROFIT** — not the number of jobs, not fairness, not speed.

The Profit Ladder Principle

High-profit jobs should get the **first chance** to occupy available slots. If a high-profit job cannot fit, we try the next one. Low-profit jobs are considered only if space remains.

Highest Profit — First pick of any slot

High Profit — Second pick of remaining slots

Medium Profit — Fills gaps if any

Low Profit — Last resort, often skipped

Key Insight

Sorting by profit descending ensures we never miss a high-profit job because of a low-profit one.

Think of it as an **auction**: the highest bidder (profit) gets the first chance at every available slot.

If the highest bidder cannot fit, we move to the next. This guarantees no "regret" — we never skip a valuable job for a less valuable one.

Why Latest Available Slot?

A job with deadline 4 can go in **slots 1, 2, 3, or 4**. Always try **slot 4 first** — this preserves earlier slots for jobs with tighter deadlines.

Visual Explanation

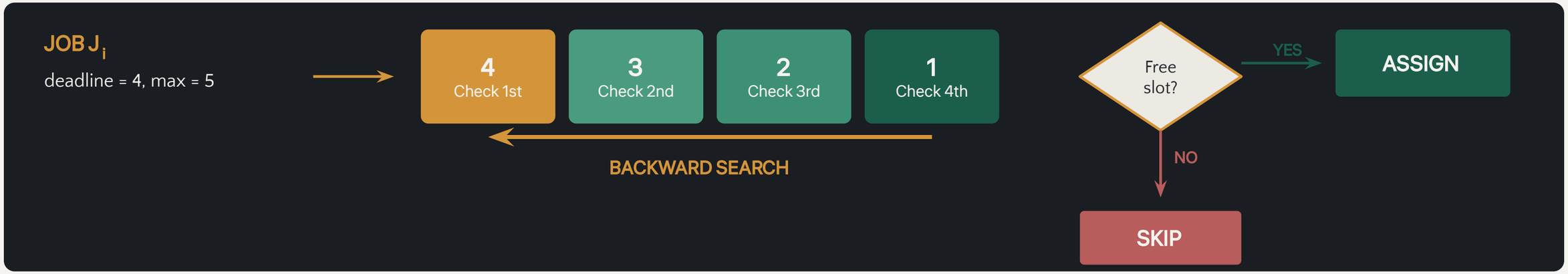
Why This Works

By placing a job in its **latest possible slot**, we keep all earlier slots free for jobs that **must** use them (jobs with earlier deadlines). This is the key to ensuring maximum feasibility.

Slot Assignment Rule

For job J_i, start checking from **min(d_i, maxDeadline)** and move **backward**. If a free slot is found, assign the job. Otherwise, **skip** it.

Slot Search Diagram



RULE IN CODE

for each job in profit-descending order:

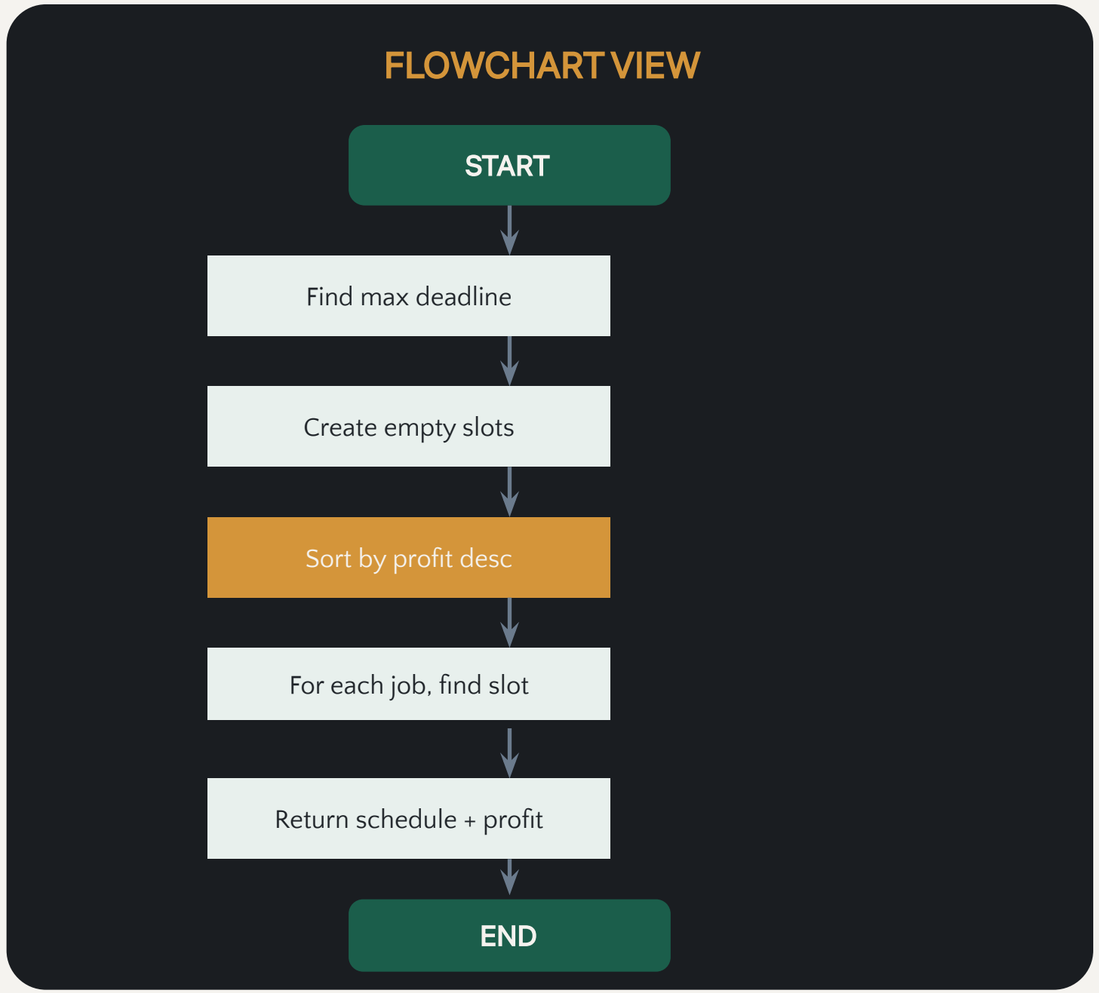
for s = min(job.deadline, maxDeadline) **downto** 1:

if slot[s] is empty:

slot[s] = job // assign and break

Algorithm Steps

1. Find the **maximum deadline** among all jobs
2. Create an **empty slot array** of that size
3. **Sort jobs by profit** in descending order
4. For each job, **search backward** from its deadline
5. **Assign** if slot is free, otherwise **skip**
6. Return the **schedule** and **total profit**



Pseudocode

JOB SEQUENCING ALGORITHM

```
// Input: Array of jobs with profit and deadline
// Output: Maximum profit and job schedule
function JobSequencing (jobs):
sort jobs by profit in descending order
maxDeadline = maximum deadline in jobs
slot = new array[maxDeadline] // all empty
totalProfit = 0
for each job in sortedJobs:
for s = job.deadline downto 1:
if slot[s] is empty:
slot[s] = job
totalProfit += job.profit
break // move to next job
return slot, totalProfit
```

Complexity at a Glance

Sorting: $O(n \log n)$ | **Slot search (naive):** $O(n^2)$ in worst case | **Total:** $O(n^2)$

Worked Example Setup

Consider the following set of **6 jobs**:

Job	Profit	Deadline	Duration
J1	100	2	1
J2	85	1	1
J3	60	3	1
J4	45	3	1
J5	40	4	1
J6	20	2	1

Maximum Deadline = 4

Therefore, we need 4 time slots

Empty Time Slots

Slot 1
EMPTY

Slot 2
EMPTY

Slot 3
EMPTY

Slot 4
EMPTY

Total Profit = **0**

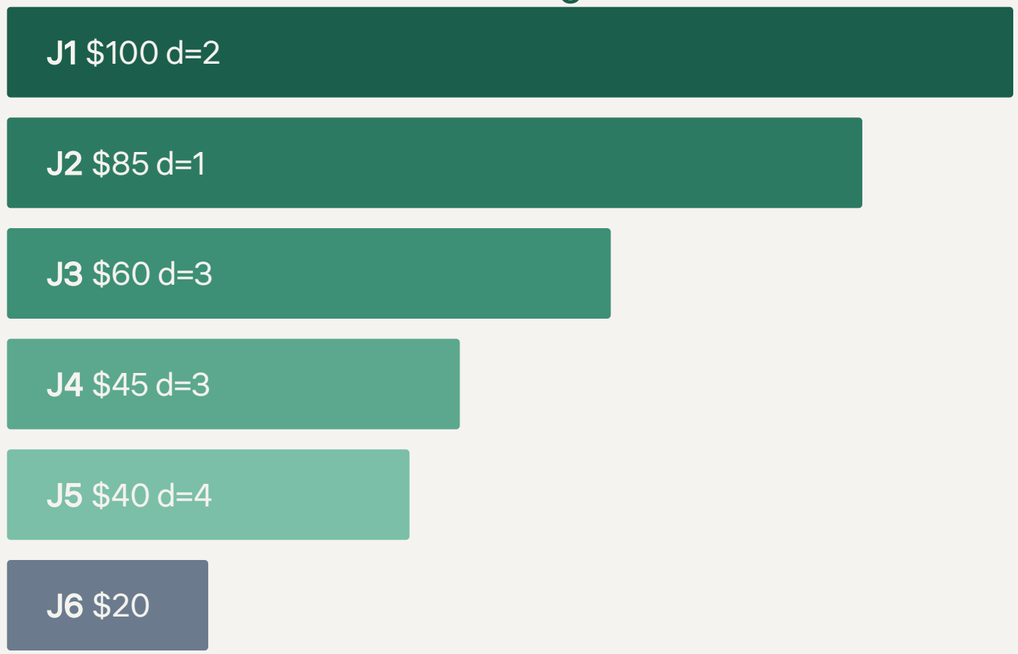
We will process jobs in order of decreasing profit and fill these slots

Step 1: Sort by Profit

Sort all jobs in **descending order of profit**. This is the order we will process them:

Order	Job	Profit	Deadline	Status
1st	J1	100	2	Pending
2nd	J2	85	1	Pending
3rd	J3	60	3	Pending
4th	J4	45	3	Pending
5th	J5	40	4	Pending
6th	J6	20	2	Pending

Profit Ranking Visual



Next: We process J1 first (highest profit), then J2, J3, J4, J5, and finally J6.

Step 0: Initial Slot Array

Before processing any jobs, all 4 slots are empty and total profit is zero.

TOTAL PROFIT

0

Ready to process jobs. Let's start with **J1** (profit = 100).

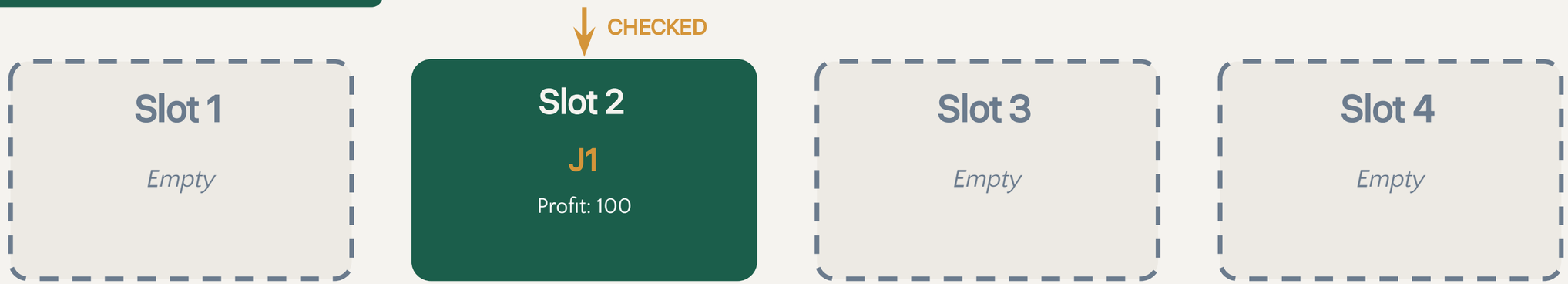
Step 1: Process J1

J1

Profit: **100** I Deadline: **2**

Check from slot **2** (deadline) backward:

Slot 2 is **free**. Place J1 in slot 2.



TOTAL PROFIT

100

Decision: J1 (deadline 2) checks slot 2 first. It is empty, so J1 is placed there. Total profit = 100.

Step 2: Process J2

J2

Profit: **85** I Deadline: **1**

CHECKED

Check from slot **1** (deadline) backward:

Slot 1 is **free**. Place J2 in slot 1.

TOTAL PROFIT

185

Decision: J2 (deadline 1) can only go in slot 1. It is empty, so J2 is placed there. Total profit = 100 + 85 = 185.

Step 3: Process J3

J3

Profit: **60** | Deadline: **3**

Check from slot **3** (deadline) backward:

Slot 3 is **free**. Place J3 in slot 3.

CHECKED

Slot 1	Slot 2	Slot 3	Slot 4
J2	J1	J3	*Empty*
Profit: 85	Profit: 100	Profit: 60	

TOTAL PROFIT

245

Decision: J3 (deadline 3) checks slot 3 first. It is empty, so J3 is placed there. Total profit = 185 + 60 = 245.

Step 4: Process J4

J4

Profit: **45** I Deadline: **3**

Check from slot **3** (deadline) backward:

Slot 3, 2, 1 are all **occupied**. Skip J4.

X X X

Slot 1
J2
Profit: 85

Slot 2
J1
Profit: 100

Slot 3
J3
Profit: 60

Slot 4
Empty

TOTAL PROFIT (unchanged)

245

SKIPPED: J4 (profit 45, deadline 3) — all slots 1–3 are occupied. No feasible slot available. Total profit remains 245.

Step 5: Process J5

J5

Profit: **40** I Deadline: **4**

Check from slot **4** (deadline) backward:

Slot 4 is **free**. Place J5 in slot 4.

CHECKED

Slot 1	Slot 2	Slot 3	Slot 4
J2	J1	J3	J5
Profit: 85	Profit: 100	Profit: 60	Profit: 40

TOTAL PROFIT

285

Decision: J5 (deadline 4) checks slot 4 first. It is empty, so J5 is placed there. Total profit = 245 + 40 = 285.

Step 6: Process J6

J6

Profit: 20 I Deadline: 2

Check from slot **2** (deadline) backward:

Slot 2 and 1 are **occupied**. Skip J6.

X X

Slot 1	Slot 2	Slot 3	Slot 4
J2	J1	J3	J5
Profit: 85	Profit: 100	Profit: 60	Profit: 40

TOTAL PROFIT (unchanged)

285

SKIPPED: J6 (profit 20, deadline 2) — slots 1 and 2 are occupied. No feasible slot. Total profit remains 285.

Final Schedule

After processing all 6 jobs, here is the **optimal schedule**:

Slot	Job	Profit	Deadline
1	J2	85	1
2	J1	100	2
3	J3	60	3
4	J5	40	4

MAXIMUM PROFIT

285

= 85 + 100 + 60 + 40

= J2 + J1 + J3 + J5

Skipped Jobs

J4 (45)

J6 (20)

Summary: 4 jobs selected (J1, J2, J3, J5), 2 jobs skipped (J4, J6). All deadlines are met. Total profit is maximized at 285.

Complete Trace Table

Step	Job	Search Path	Decision	Action	Profit
0	–	Initial state	–	–	0
1	J1	Slot 2 (free)	Available	SELECT	100
2	J2	Slot 1 (free)	Available	SELECT	185
3	J3	Slot 3 (free)	Available	SELECT	245
4	J4	Slots 3, 2, 1 (all full)	Blocked	SKIP	245
5	J5	Slot 4 (free)	Available	SELECT	285
6	J6	Slots 2, 1 (all full)	Blocked	SKIP	285

LEGEND

Selected (profit increases)

Skipped (no feasible slot)

Bold amber = profit increased | Gray = profit unchanged

What If We Choose Wrong?

Both rules are essential. Violating either one leads to **suboptimal profit**. Let's see what happens:

MISTAKE 1

Sort by Deadline (not Profit)

Low-profit jobs with early deadlines get scheduled first, blocking high-profit jobs.

Result: Lower total profit because high-value jobs are excluded.

MISTAKE 2

Assign Earliest Slot (not Latest)

Placing jobs in early slots wastes them on jobs that could use later slots.

Result: Tight-deadline jobs have no slots left, reducing feasible set.

CORRECT

Both Rules Together

1. **Sort by profit descending** ensures high-value jobs are considered first.
2. **Assign to latest free slot** preserves early slots for tight-deadline jobs.

Correctness Intuition

The greedy strategy works because it never makes a **locally suboptimal** choice that could lead to a globally worse solution.

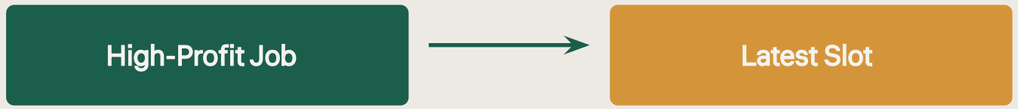
High-Profit Jobs Protected

By processing high-profit jobs first, we ensure they get the best chance at a feasible slot. We never "use up" slots on low-profit jobs before checking if a high-profit job needs one.

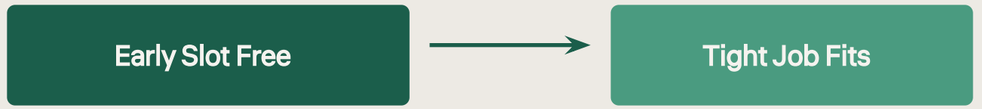
Early Slots Preserved

By placing jobs in their latest possible slot, we keep early slots open for jobs with tighter deadlines. This maximizes the number of jobs that can potentially be scheduled.

Visual Proof Sketch



High-profit job placed late = early slots stay free for tight-deadline jobs



Tight-deadline job finds an early slot = both jobs are scheduled

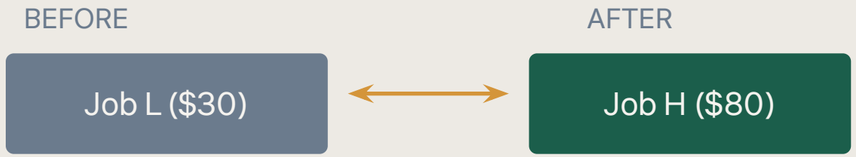
Exchange Argument

If an optimal schedule omits a feasible high-profit job, we can **exchange** it with a lower-profit job without decreasing total profit.

The Exchange Process

1. Suppose optimal schedule **S** contains a lower-profit job **L**
2. A higher-profit job **H** is feasible but not in **S**
3. **Replace L with H** in the same slot (or a later one)
4. New schedule is still feasible and profit **does not decrease**

VISUAL: THE SWAP



Profit: +50 (or same)

By repeatedly applying this exchange, any optimal schedule can be transformed into the greedy schedule without decreasing profit.

Conclusion: The greedy algorithm produces a schedule with profit at least as high as any optimal schedule. Therefore, the greedy schedule is **optimal**.

Complexity Analysis

Naive Version

Operation	Complexity	Notes
Sorting jobs	O(n log n)	Merge/Quick sort
Slot searching	O(n^2)	Worst case scan
Total Time	O(n^2)	Dominates sorting

Space Complexity: O(n) for the slot array

Optimized (DSU) Version

Using **Disjoint Set Union (DSU)** with path compression:

Find operation: O(alpha(n)) ~ O(1)

Union operation: O(alpha(n)) ~ O(1)

Slot search becomes nearly constant time per job.

Total Time: O(n log n)

Space: O(n)

Time Complexity Comparison

Common Mistakes

Watch out for these common pitfalls!

✗ Sorting by deadline only

This maximizes job count, not profit

✗ Assigning earliest slot

Wastes early slots on loose-deadline jobs

✗ Forgetting to skip impossible jobs

Must handle the case where no slot fits

✗ Scheduling after deadline

Job must finish ON or BEFORE deadline

✗ Maximizing number of jobs instead of profit

The goal is profit maximization — a single high-profit job may be better than multiple low-profit ones

Correct Approach Checklist

- ✓ Sort by **profit descending**
- ✓ Place in **latest free slot**
- ✓ **Skip** if no slot fits

KEY TAKEAWAYS

1. **Job Sequencing** is a classic greedy scheduling problem where each job has a profit and a deadline.

2. **Correct Strategy:** Sort by profit descending, then place each job in the **latest free slot** before its deadline.

3. **Naive Complexity:** $O(n^2)$ time, O(n) space. **Optimized (DSU):** $O(n \log n)$ time.

4. The greedy choice is proven optimal via the **exchange argument** — never regret a locally optimal decision.

THANK YOU FOR YOUR ATTENTION